



SJF vs FCFS Scheduling

Shortest Job First (non-preemptive) compared against First-Come, First-Served CPU scheduling

SJF EXECUTION



FCFS EXECUTION



PROCESS BLOCKS SIZED BY BURST TIME – SAME FIVE PROCESSES, TWO ORDERINGS – BOTH FINISH AT T=17

5

PROCESSES

17

TOTAL BURST (UNITS)

2

ALGORITHMS COMPARED

SJF

LOWER AVG. WAIT

COURSE CSE 402 – Operating System Lab

EXPERIMENT Comparison of SJF and FCFS Scheduling

REPOSITORY github.com/ashadsetu/CSE-402

SUBMITTED BY

MD Asadullah

Reg. No 23101126 · Section C1
4th Year, 1st Semester

SUBMITTED TO

Atia Rahman Orthi

Lecturer, Department of CSE
University of Asia Pacific

PROBLEM

Schedule five processes with non-preemptive SJF, then again with FCFS, and compare average Turnaround Time (TAT) and Waiting Time (WT) to see which performs better on this workload.

PID	AT	BT
P1	2	5
P2	3	1
P3	0	2
P4	1	3
P5	1	6

Non-preemptive SJF scans every process that has already arrived (`AT[i] <= time`) and dispatches whichever has the smallest burst time.

CODE — SJF

```
pid = ["P1", "P2", "P3", "P4", "P5"]
AT = [2, 3, 0, 1, 1]
BT = [5, 1, 2, 3, 6]

n = 5
completed = [False] * n
CT, TAT, WT = [0]*n, [0]*n, [0]*n
time, finished, sequence = 0, 0, []

while finished < n:
    idx, min_bt = -1, 9999
    for i in range(n):
        if not completed[i] and AT[i] <= time:
            if BT[i] < min_bt:
                min_bt, idx = BT[i], i

    if idx == -1:
        time += 1
    else:
        time += BT[idx]
        CT[idx] = time
        completed[idx] = True
        finished += 1
        sequence.append(pid[idx])

for i in range(n):
    TAT[i] = CT[i] - AT[i]
    WT[i] = TAT[i] - BT[i]

avg_tat_sjf = sum(TAT) / n
avg_wt_sjf = sum(WT) / n
print("Execution Sequence:", " ".join(sequence))
```

■ RESULT – SJF

PID	AT	BT	CT	TAT	WT
P1	2	5	11	9	4
P2	3	1	6	3	2
P3	0	2	2	2	0
P4	1	3	5	4	1
P5	1	6	17	16	10

```
Sequence: P3 -> P4 -> P2 -> P1 -> P5
Average TAT = 6.8      Average WT = 3.4
```

FCFS

The same five processes, re-scheduled by arrival order instead of burst time.

■ CODE – FCFS

```
pid = ["p3", "p4", "p5", "p1", "p2"]
AT = [0, 1, 1, 2, 3]
BT = [2, 3, 6, 5, 1]

CT, TAT, WT = [], [], []
ct = 0

for i in range(5):
    if ct < AT[i]:
        ct = AT[i]
    ct += BT[i]
    CT.append(ct)

for i in range(5):
    tat = CT[i] - AT[i]
    TAT.append(tat)
    WT.append(tat - BT[i])

avg_tat_fcfs = sum(TAT) / 5
avg_wt_fcfs = sum(WT) / 5
print("Execution Sequence:", " -> ".join(pid))
```

■ RESULT – FCFS

PID	AT	BT	CT	TAT	WT
p3	0	2	2	2	0
p4	1	3	5	4	1
p5	1	6	11	10	4
p1	2	5	16	14	9
p2	3	1	17	14	13

```
Sequence: p3 -> p4 -> p5 -> p1 -> p2
Average TAT = 8.8      Average WT = 5.4
```

VERDICT

SJF	FCFS
Avg TAT 6.8 · Avg WT 3.4	Avg TAT 8.8 · Avg WT 5.4

SJF wins on both averages for this workload — lower TAT (6.8 vs 8.8) and lower WT (3.4 vs 5.4).

DISCUSSION

- **The arrival check is what makes SJF work.** Grabbing the globally-shortest burst is wrong if that process hasn't arrived yet — `AT[i] <= time` keeps the choice limited to what's actually in the ready queue. We only saw why it mattered after tracing P5 getting skipped early for not having shown up.
- **The convoy effect is visible in the numbers.** p5, the largest job, arrives third and blocks p1 and p2 behind it under FCFS even though both are shorter. That ordering alone pushes average WT from 3.4 to 5.4 — nothing about the processes changed, only who went first.
- **SJF's average hides an unfair extreme.** P5 has the worst individual wait in the whole report (10 units) — SJF gets a lower average precisely by always making the longest job step aside. Scaled to a real system with a constant stream of short jobs, that bias has no natural stopping point: a long process can be passed over indefinitely. That's **starvation**, the standard critique of pure SJF — most real schedulers add **aging** so priority climbs the longer a process waits.
- **SJF also assumes information FCFS doesn't need.** Non-preemptive SJF needs burst time known in advance, which a real OS can only estimate from history. Our numbers look clean here only because the burst times were handed to us outright.

CONCLUSION

- Both algorithms were traced by hand, then verified in Python — the two agree once the FCFS arrival-time bug was fixed.
- **SJF minimizes the average** on this workload, matching the textbook result, but the process that made its average look good is the same one it made wait longest.
- **FCFS is worse on average but fair by construction** — arrival order guarantees every process eventually runs, however long the jobs behind it are.
- The two aren't simply "better" and "worse": **SJF optimizes throughput, FCFS optimizes fairness.** Which matters more depends on the system being built.

On this data set **SJF** wins average TAT/WT (6.8 / 3.4 vs 8.8 / 5.4) — FCFS wins on guaranteeing every process gets its turn.